

Symmetry Reduction in the Detailed Balance Checker: Expected Speedup, Feasibility, and the Black-Box Problem

1 The proposed method

For a translation-invariant algorithm on an $L \times L$ periodic lattice, all nine translated copies of a state pair (s, s') satisfy the same detailed balance (DB) condition. A three-line argument shows: if DB holds for (s, s') , it holds for $(g \cdot s, g \cdot s')$ for every translation g , and vice versa. Checking one representative per family of nine therefore suffices. With full D4 symmetry (8 rotations and reflections $\times$ 9 translations = $|G| = 72$), the reduction is 72-fold.

The runtime bottleneck is `FullSimplify`, called on symbolic DB expressions that `FastChecker` could not resolve. At `NGrid=3` ($N = 504$), roughly 300 such calls at ~ 0.5 s each across 4 kernels give the observed ~ 3 min runtime. Orbit reduction to $1/|G|$ of pairs would reduce these calls proportionally.

(NGrid = 3, $N=504$)	No symmetry	Translations ($ G =9$)	D4+trans. ($ G =72$)
Pairs to check	126,756	14,084	$\approx 1,760$
FullSimplify calls	~ 300	~ 33	~ 4
Estimated runtime	3 min	~ 20 s	~ 2 s

For `NGrid=6` ($N \approx 100,000$) the full check requires $\sim 5 \times 10^9$ pairs and is infeasible. With $|G|=288$, this reduces to $\sim 1.7 \times 10^7$ — potentially tractable.

2 The black-box problem

The checker treats algorithms as black boxes: it supplies random-number inputs and records symbolic outputs. The orbit reduction is only valid if $T(g \cdot s \rightarrow g \cdot s') = T(s \rightarrow s')$ for all pairs and all $g \in G$. This document asks: can this G-invariance property be verified in the same black-box way, without reading or interpreting source code?

Why the existing hash method is watertight but unhelpful here

The current checker deduplicates symbolic expressions using `Hash[]`: it skips re-evaluating a DB expression only when an identical expression (same hash, same syntactic form) has already been seen. This is genuinely watertight — it makes no assumption about G-invariance and never skips a pair unless the work is demonstrably redundant.

The orbit reduction is a fundamentally different claim: it would skip a pair because a *G-equivalent* pair was already checked — not because the expressions were observed to be identical. For this to be valid, G-equivalent pairs must produce identical DB expressions. But G-equivalent pairs reference different site indices and different neighbor-list orderings, so they produce syntactically different symbolic expressions even when mathematically equal. They hash differently. The existing deduplication does not group them (this is why the $\sim 3,000$ unique expressions at `NGrid=3` do not already collapse to ~ 42).

To confirm that G-equivalent pairs produce mathematically equal expressions, you would need `FullSimplify` on each pair in the orbit — which costs as much as the DB check itself. The verification tool is no cheaper than the thing it is meant to enable.

The single-microstate counterexample

Suppose an algorithm is G-invariant for every state *except* one specific microstate s^* , where the transition probabilities are slightly wrong. A black-box checker that tests only a sample of states could never detect this, because the probability of including s^* in a small sample from $N = 504$ (or $N = 100,000$) states is tiny.

The only guarantee comes from testing *every* state. But testing all N states means computing symbolic DB expressions for all $N(N - 1)/2$ pairs — which is exactly the cost of the full check. There is no intermediate regime: either you accept an unverified assumption, or you pay the full cost.

This is not a hypothetical concern. The `vmmc.biased.wl` algorithm fails G-invariance specifically because rightward-displaced pairs behave differently from the rest, and a checker that happened not to examine those pairs would miss the violation entirely.

The fundamental impossibility

Any reliable black-box G-invariance check must, in the worst case, test all N states and all $N(N - 1)/2$ pairs between them. This is not a limitation of cleverness; it follows from the counterexample above: no sampling-based test can exclude the possibility that one untested state violates G-invariance. Reliable G-invariance verification therefore costs $O(N^2)$ symbolic evaluations — the same asymptotic cost as the full DB check. The speedup the orbit reduction provides is exactly consumed by the cost of verifying that the speedup is valid.

3 Alternative approaches worth considering

Probabilistic verification with known risk. Check G-invariance for a random sample of state pairs and accept a small residual probability of false negatives. For k randomly sampled pairs, a violation affecting fraction f of all pairs is missed with probability $(1 - f)^k$. With $k = 100$ samples and $f = 0.01$ (1% of pairs affected), the miss probability is $(0.99)^{100} \approx 0.37$ — unacceptably high for a correctness checker. This approach is appropriate for informal exploration but not for publication-quality checking.

Cross-checking within orbits. Check the orbit representative *and* one randomly chosen orbit member, at a total cost of $2/|G|$ per orbit. If the two DB outcomes disagree, G-invariance has demonstrably failed for that orbit. If they agree, either G-invariance holds or both checked members happen to have the same (wrong) DB status. This halves the speedup factor ($|G|/2$ instead of $|G|$) while catching gross G-invariance failures. It does not provide a correctness guarantee, but it converts an invisible false negative into a detectable inconsistency for the most common failure mode.

Algebraic canonicalisation before hashing. The reason G-equivalent pairs hash differently is that the symbolic expressions reference absolute site indices. If the `seqBernoulli` output were normalised to replace absolute sites with relative positions before hashing — expressing all couplingJ atoms in terms of the seed particle’s position rather than the lattice origin — G-equivalent pairs would produce identical expressions and identical hashes. This would make the existing hash deduplication automatically capture G-orbits, with no separate verification step needed. The normalisation step would be applied to every expression as it is generated, adding constant overhead per expression but no asymptotic cost. This is the most promising avenue and

deserves further investigation, but implementing it requires understanding the structure of the seqBernoulli output (which expressions reference sites, how to re-express them in seed-relative coordinates), which is non-trivial.

Algorithm metadata interface. Require algorithm files to declare their symmetry group explicitly (`$symmetryGroup = "D4Translation"`). The checker uses this declaration to apply the orbit reduction and takes the declaration at face value. This is black-box in the sense of not requiring source-code parsing, but the correctness burden shifts to the algorithm author: an incorrect declaration introduces undetected false negatives.

4 Conclusion: is the method feasible?

At NGrid=3: No. The orbit reduction gives $\sim 9\times$ speedup on FullSimplify calls, cutting 3 min to ~ 20 s for translations (or ~ 2 s for D4). But reliable G-invariance verification has the same asymptotic cost as the full check. The 3 min runtime is already acceptable, and the implementation introduces seven correctness conditions and the risk of silent false negatives. Not worth implementing.

At NGrid=6 and above: The full check is infeasible regardless. The orbit reduction is a prerequisite for the checker to run at all. At this scale the question is not “is the speedup worth it?” but “can we verify G-invariance at all?” The only practical option is a brief source-code inspection (three specific checks described below), accepted as a necessary precondition rather than an automated step.

For vmc_lattice.wl-style algorithms, the source-code check takes under five minutes and reduces to three observations: (1) seed selection is `RandomChoice[occupied]` with no position-dependent weighting; (2) displacement selection is `RandomChoice[$displacements]` where `$displacements` is a fixed list that does not depend on the current state; (3) the energy function calls `couplingJ[a, b, d2]` where `d2` is computed using the minimum-image periodic distance formula $\min(\Delta r, L - \Delta r)^2 + \min(\Delta c, L - \Delta c)^2$.

Overall verdict: The orbit reduction is worth implementing *only* for $\text{NGrid} \geq 5$, and *only* if the source-code precondition is accepted. The pure black-box ideal — verifying G-invariance without reading source code and without paying the full DB check cost — is unachievable. Algebraic canonicalisation of the seqBernoulli output is the one avenue that might bridge this gap; if it can be implemented, it would make G-invariance verification automatic and free.